MITTUNIVERSITETET
Laboration: L3 Minnet
Laborant: Hussein Alshaikhshaker
Kurs: Operativ System (HT24-DT201G)
Datum: 2024-11-24

Resurser

Operativ System	Windows 11 Home 23H2
Primärminne *(GB)*	8
Sekundärminne (GB)	377

Test

Tabellen nedan presenterar resultaten från tester där maximal kapacitet av primärminnet undersöktes. Programmet som användes allokerar successivt minne i primärminnet och virtuella minnet tills resurser tar slut.

Test	*Platform*	*Chunk storlek*	*Konfiguration*	*Antal blocks*	*Tid* ***(Millisekunder)***	*Allokerat minne* ***(GB)***	*Task manager* ***(GB)***	*VS* ***(GB)***	*Ledigt minne* ***(GB)***
1	32-bits	1 GiB	LAA_on	2	4	2	10.6/18.7	2	2.9
2	32-bits	1 GiB	LAA_off	1	3	1	8.9/18.7	1	1.8
3	32-bits	1 KiB	LAA_on	4 044 247	5512	3.85689	11.9/18.2	4	2.5
4	32-bits	1 KiB	LAA_off	2 014 210	2435	1.9209	10.0/18.2	2	2.8
5	64-bits	1 GiB	LAA_on	21	1621	21	31.0/31.7	22	3.0
6	64-bits	1 GiB	LAA_off	1	5	1	11.0/18.7	1	2.3
7	64-bits	1 KiB	LAA_on	-	-	-	31.5/31.7	24.1	2.7
8	64-bits	1 KiB	LAA_off	1 991 588	2417	1.89933	10.0/18.2	2	2.4

Analys

Programmets egna utskrifter skriver ut explicit hur mycket programmets tråd allokerade i minnet. Task manager representerar den totala använda primärminneskapacitet samt virtuella minneskapaciteten som används för tillfället.

Minnesadressen som programmet allokerar varierar beroende på vilken plattform programmet kompileras på. 32-bitars system har som standard tillgång till 2GB fysiskt minne där adressrymden kan nå upp till 2^32 adresser. Om LAA är aktiverat kan applikationer använda upp till 4GB i stället för standardgränsen på 2GB. [1]

Adressrymden för 64-bitars system är 2^64, vilket teoretiskt sett skulle kunna stödja upp till 16 exabytes primärminne. Men i praktiken finns det ingen dator idag som har hårdvara som kan hantera så mycket primärminne. Datorer idag har betydligt mycket mindre primärminne an vad 64-bitars adressrymd skulle kunna stödja. [3]

Large Address Aware	*Plattform*	*Chunk storlek **(B)***	*Adress **(HEX) (nr)***
Off	32-bits	1 KiB	0049EA18 **(1)**
On	32-bits	1 KiB	77D6EA18 **(2)**

Tabellen visar sista allokerade minnesadresserna efter utförda körningar av programmet.

Adresserna ovan visar då hur LAA påverkar allokeringen. Om LAA är avslagen kommer operativsystemet att tillåta programmet allokera addresser till en viss gräns(2GB) för användarprocesser. Om LAA är på så kommer operativsystemet att ge programmet tillgång till mer adresser.

Den virtuella adressrymden för varje process kan vara mindre eller större än fysiska minnet. Om trådarna i en process begär att använda mer fysiskt minne än vad som finns tillgängligt, hanteras det med hjälp av paging som i sin tur använder sekundärminnet. [2]
I test #7 ser vi att programmet kraschade när den hade nått upp till

31.5GB allokerat minne. Denna kapacitet överskrider tillgängligt fysiskt och virtuellt minne.

I Test #3 allokerades 4GB minne med LAA på och chunkstorlek på 1KiB, i test #1 allokerades 2GB minne med LAA också på och chunkstorlek på 1GiB. Detta beror på fragmentering i minnet. Antalet blocks allokerade i test #3 var betydligt många mer än antalet blocks allokerade i test #1 pga chunkstorleken. Med allokering av mindre chunkstorlekar är det mer sannolikt att MMU hittar sammanhängande minneluckor för arrayerna. Medan vid större chunkstorlek kan det vara svårt för MMU att hitta luckor i fysiska minnet som räcker för arrayer på 1GiB var. [4] [5]

Extern fragmentering uppstod under mina undersökninar. Minneskompaktering motverkade fragmenteringen och räddade lite mer utrymme i fysiska minnet. Minnekompakteringen fungerar på så sätt att den skjuter upp allokerat minne mot ena enden av fysiska minnet och det lediga minnet sjukts upp mot det andra hållet, på så sätt får vi större lediga utrymmen i fysiska minnet. [4] [5]

I test #1 ser vi att chunkstoreleken påverkade resultatet av antalet block allokerade, den maximala allokerade minneskapaciteten nådde 2GB. I task manager ökade (committed memory) med 2GB trots att LAA var på. Den nådde aldrig 3 eller 4 GB sannolikt pga att det inte fanns tillräckligt med sammanhängande ledigt utrymme för att allokera en chunk på 1 GiB. Det kanske fanns tillräckligt med totalt ledigt utrymme för att rymma mer än 1 GiB, men det var uppdelat i mindre delar.

I test #1 ser vi att chunkstoreleken påverade antalet allokerade minneblock. Trots att LAA var aktiverat nåddes den högsta allokerade minneskapaciteten på 2GB. I task manager ökade ”committed memory” med 2GB, även fast LAA var aktiverat. Programmed nådde aldrig 3- eller 4GB, troligen på grund av att det fysiska minnet inte hade tillräckligt med sammanhängande lediga utrymmen för att allokera så mycket minne.

Källhänvisningar

[1] [Online]. Available: https://learn.microsoft.com/en-us/cpp/build/reference/largeaddressaware-handle-large-addresses?view=msvc-170.

[2] [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/memory/virtual-address-space-and-physical-storage.

[3] [Online]. Available: https://www.geeksforgeeks.org/32-bit-vs-64-bit-operating-systems/.

[4] [Online]. Available: https://apachepersonal.miun.se/~jimahl/rec/DT201G/8_fysiskt_minne.mp4.

[5] ”Operating System concepts, 10th ed, Chapt 9 Main Memory,” 2018.